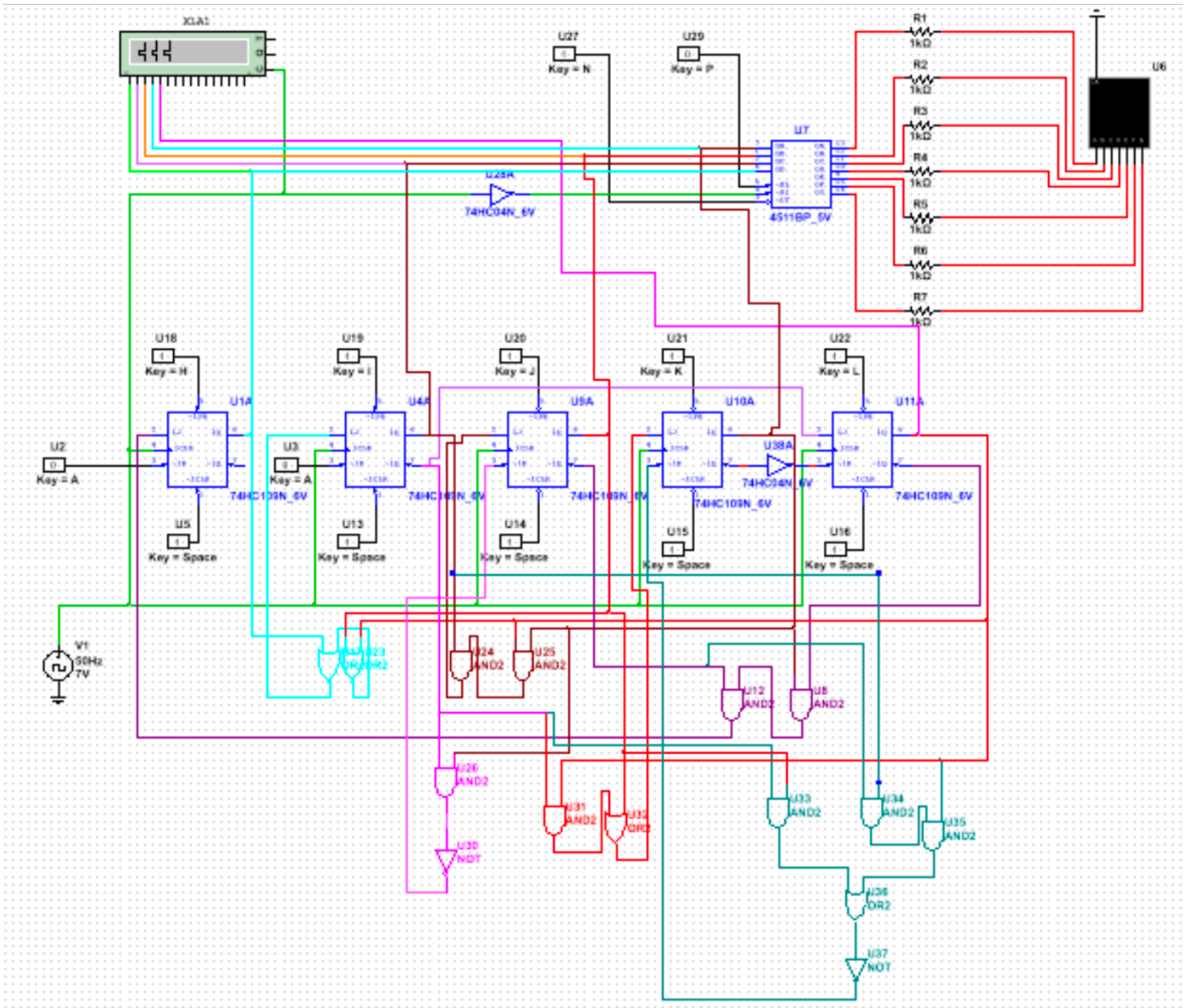
Overview:



Above is the final Multisim set up of a circuit featuring sequential logic design. The circuit cycles through numbers in the order: [redacted]. In total 5 JK flip flops, 9 AND gates, 4 OR gates and 3 NOT gates were used to achieve this goal. Four of the JK flip flops act as binary counters which are decoded before being passed onto the 7-segment display and the fifth one is used as a counter which can keep track of repeating numbers. Incorporated are reset switches, ensuring that the circuit starts in a predictable state each time. This prevents the circuit from entering undesirable loops.

Process:

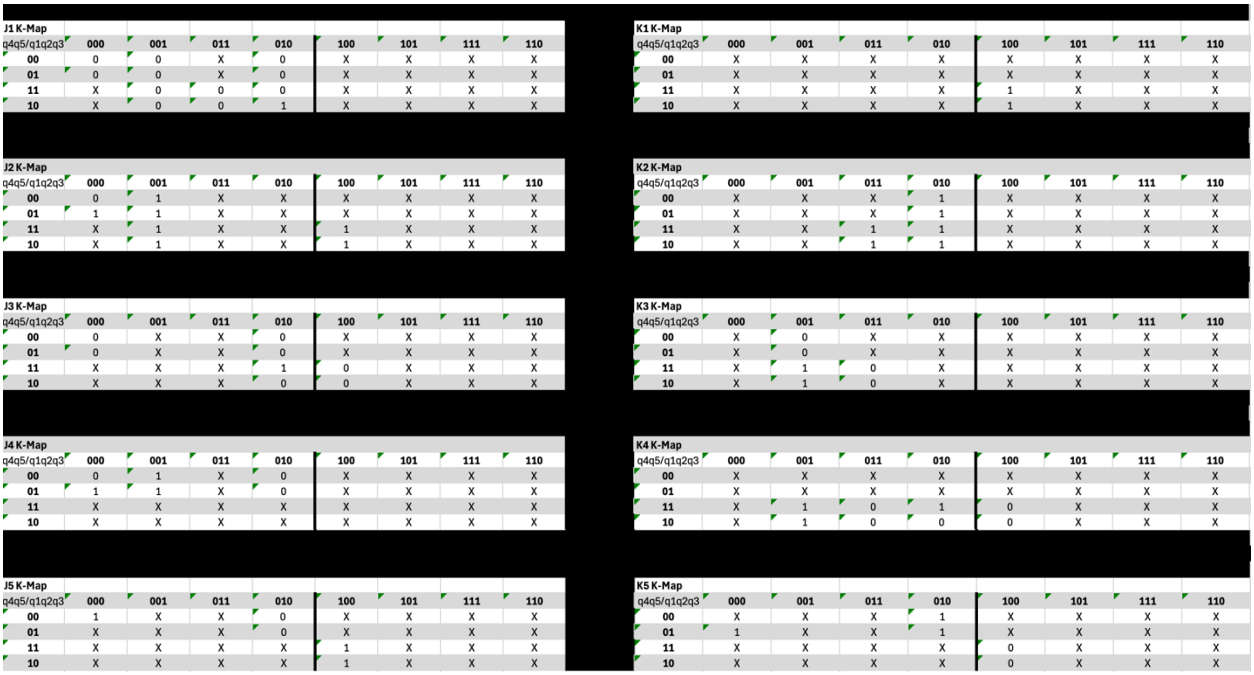
First the desired digits were converted from Arabic numerals to binary, giving a foundation for the state transition table. Following, the state transition table was made by assigning current and next states for both the counter and the binary representation bits. Then, the states of the JK circuit were found by using the following truth and excitement tables.

JK Truth Table				JK Excitement Table			
J	**K**	**Output**		**Q_C**	**Q_N**	**J**	**K**
0	0	Hold		0	0	0	X
0	1	Reset		0	1	1	X
1	0	Set		1	0	X	1
1	1	Toggle		1	1	X	0

Using these tables, it was easy to determine what the necessary states of J and K would be to reach the next state in the cycle, where 1 represents ON, 0 represents OFF and X represents "Don't care" (a situation where the value of that square does not impact the output).

Current State				Current Memory	Next State				Next Memory	Flip-Flop Inputs									
q1	q2	q3	q4	q5	Q1	Q2	Q3	Q4	Q5	J1	K1	J2	K2	J3	K3	J4	K4	J5	K5
0	1	0	0	X	0	0	0	0	0	0	X	X	1	0	X	0	X	0	1
0	0	0	0	0	0	0	0	0	1	0	X	0	X	0	X	0	X	1	X
0	0	0	0	1	0	1	0	1	0	0	X	1	X	0	X	1	X	X	1
0	1	0	1	0	1	0	0	1	X	1	X	X	1	0	X	X	0	X	X
1	0	0	1	X	0	1	0	1	1	X	1	1	X	0	X	X	0	1	0
0	1	0	1	1	0	0	1	0	X	0	X	X	1	1	X	X	1	X	X
0	0	1	0	X	0	1	1	1	X	0	X	1	X	X	0	1	X	X	X
0	1	1	1	X	0	0	1	1	X	0	X	X	1	X	0	X	0	X	X
0	0	1	1	X	0	1	0	0	X	0	X	1	X	X	1	X	1	X	X

From this state transition table, each J and K input was individually added to its own K-map. This allows for Boolean expressions and a physical circuit to be made. The K-maps are as follows.



J1 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	0	X	0	X	X	X	X
01	0	0	X	0	X	X	X	X
11	X	0	0	0	X	X	X	X
10	X	0	0	1	X	X	X	X

K1 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	X	X	X	X	X
01	X	X	X	X	X	X	X	X
11	X	X	X	X	1	X	X	X
10	X	X	X	X	1	X	X	X

J2 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	1	X	X	X	X	X	X
01	1	1	X	X	X	X	X	X
11	X	1	X	X	1	X	X	X
10	X	1	X	X	1	X	X	X

K2 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	1	X	X	X	X
01	X	X	X	1	X	X	X	X
11	X	X	1	1	X	X	X	X
10	X	X	1	1	X	X	X	X

J3 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	X	X	0	X	X	X	X
01	0	X	X	0	X	X	X	X
11	X	X	X	1	0	X	X	X
10	X	X	X	0	0	X	X	X

K3 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	0	X	X	X	X	X	X
01	X	0	X	X	X	X	X	X
11	X	1	0	X	X	X	X	X
10	X	1	0	X	X	X	X	X

J4 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	1	X	0	X	X	X	X
01	1	1	X	0	X	X	X	X
11	X	X	X	X	X	X	X	X
10	X	X	X	X	X	X	X	X

K4 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	X	X	X	X	X
01	X	X	X	X	X	X	X	X
11	X	1	0	1	0	X	X	X
10	X	1	0	0	0	X	X	X

J5 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	1	X	X	0	X	X	X	X
01	X	X	X	0	X	X	X	X
11	X	X	X	X	1	X	X	X
10	X	X	X	X	1	X	X	X

K5 K-Map

q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	1	X	X	X	X
01	1	X	X	1	X	X	X	X
11	X	X	X	X	0	X	X	X
10	X	X	X	X	0	X	X	X

Boolean Expressions:

A 5 input K-map looks slightly different to a 4 input K-map. There is a left and right side of the K-map to account for when one of the variables is 1/0. The left and right sides of the map can be stacked on top of one another, and any rectangles can be expanded upwards or downwards so long as they keep size 2^n.

J1 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	0	X	0	X	X	X	X
01	0	0	X	0	X	X	X	X
11	X	0	0	0	X	X	X	X
10	X	0	0	1	X	X	X	X

The J1 K-map features two rectangles, one covering the squares q2=1 and q3=0 and one where q2=q3=0. The second rectangle is a byproduct of Boolean expression, which just happens to encompass it. $q_4\overline{q_5}\,\overline{q_3}$ is the Boolean for J1 as q4 and q5 eliminate all but the bottom row and squares which have q3=1 also are equal to 0.

K1 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	X	X	X	X	X
01	X	X	X	X	X	X	X	X
11	X	X	X	X	1	X	X	X
10	X	X	X	X	1	X	X	X

The only two squares in the K1 K-map which have an impact on the circuit are both equal to 1. This means that wiring the K1 input to always on is sufficient and the simplest design choice.

J2 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	1	X	X	X	X	X	X
01	1	1	X	X	X	X	X	X
11	X	1	X	X	1	X	X	X
10	X	1	X	X	1	X	X	X

The J2 K-map only has 1 square which is 0, it also happens to be when all the inputs are equal to 0. The Boolean expression $q_1 + q_3 + q_5$ encompasses all the required spaces here. Other combinations such as replacing q1 with q4 could work, it just comes down to preference.

K2 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	1	X	X	X	X
01	X	X	X	1	X	X	X	X
11	X	X	1	1	X	X	X	X
10	X	X	1	1	X	X	X	X

Similar to the K1 K-map, none of the squares in the K2 K-map are 0, so it can always be high.

J3 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	X	X	0	X	X	X	X
01	0	X	X	0	X	X	X	X
11	X	X	X	1	0	X	X	X
10	X	X	X	0	0	X	X	X

The J3 K-map has one rectangle (stacked). The only row which has no zeros is when q4 and q5 are high. This leads to the Boolean expression $q_2 q_4 q_5$.

K3 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	0	X	X	X	X	X	X
01	X	0	X	X	X	X	X	X
11	X	1	0	X	X	X	X	X
10	X	1	0	X	X	X	X	X

The K3 K-map also has 1 rectangle because of the stacking feature. This extra leniency allows for the Boolean expression to only be two terms: $\overline{q_2} q_4$.

J4 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	0	1	X	0	X	X	X	X
01	1	1	X	0	X	X	X	X
11	X	X	X	X	X	X	X	X
10	X	X	X	X	X	X	X	X

The J4 K-map has two rectangles, with one being connected to q3 and the other being related to q2 and q5. This results in the Boolean expression: $\overline{q_2} q_5 + q_3$.

K4 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	X	X	X	X	X
01	X	X	X	X	X	X	X	X
11	X	1	0	1	0	X	X	X
10	X	1	0	0	0	X	X	X

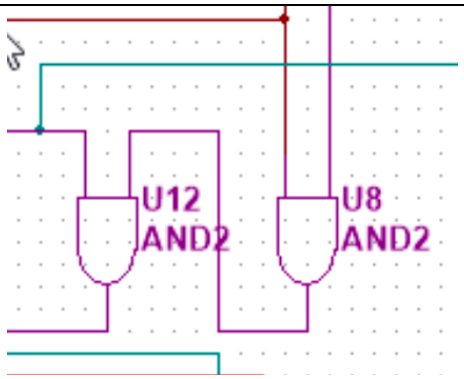
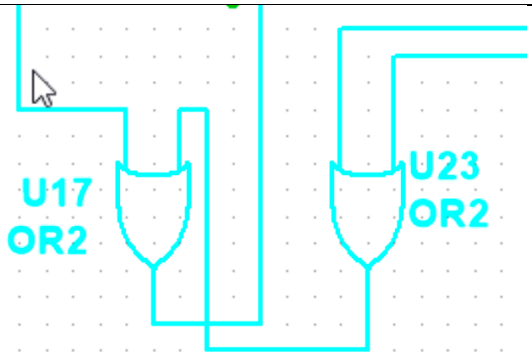
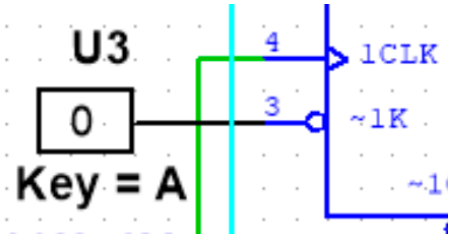
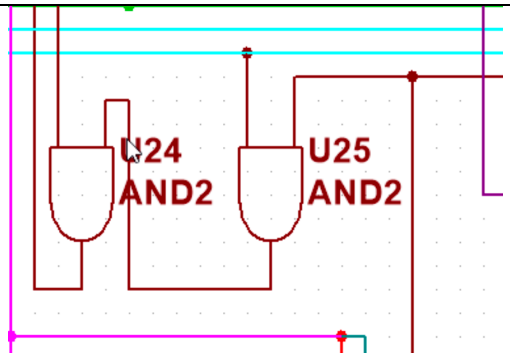
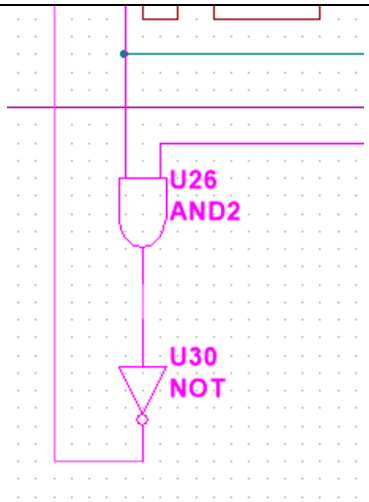
The K4 K-map closely represents a XOR gate, but the 0 in the 01010 position prevents this gate from being used. This leads to a slightly more complex expression: $\overline{q_2} q_3 + q_5 q_2 \overline{q_3}$.

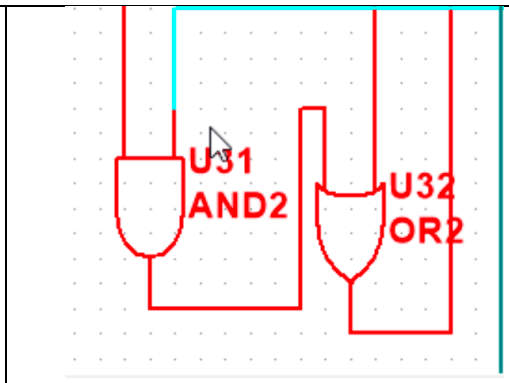
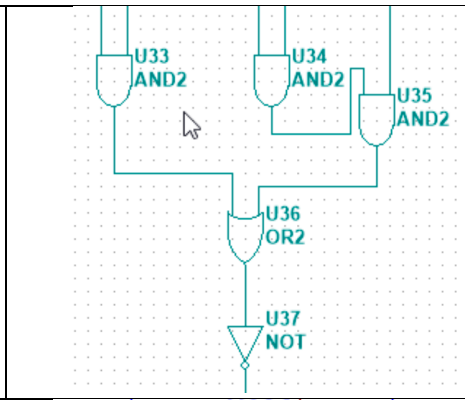
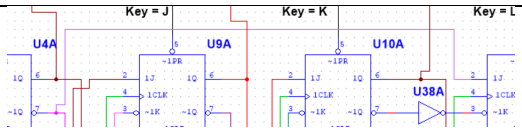
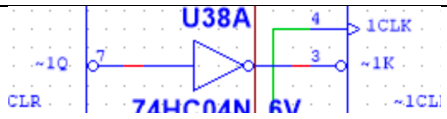
J5 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	1	X	X	0	X	X	X	X
01	X	X	X	0	X	X	X	X
11	X	X	X	X	1	X	X	X
10	X	X	X	X	1	X	X	X

The J5 K-map is one rectangle. It can be easily represented with $\overline{q_2}$.

K5 K-Map								
q4q5/q1q2q3	000	001	011	010	100	101	111	110
00	X	X	X	1	X	X	X	X
01	1	X	X	1	X	X	X	X
11	X	X	X	X	0	X	X	X
10	X	X	X	X	0	X	X	X

The K5 K-map is also one rectangle. It can be represented with either $\overline{q_4}$ (pictured here) or $\overline{q_1}$.

Flip-Flop	J	K
1		
2		
3		

4		
5		

The table above shows the gates which were used to achieve the Boolean expressions above. Some of these would be made simpler with 3 input gates, but those are not available, so staggered setups were used. The gates in lab which will be used are 74HC109N, which are J/NOT K inputs. The Booleans derived rely on JK inputs, so to fix this, a NOT gate was added to q3, q4, and q5. q1 and q2 were set to low instead of high, and q4 could have been connected to q4 instead of $\overline{q4}$, but since nothing was connected to $\overline{q4}$, so just adding a NOT gate improves readability.

Design Choices

Throughout the set-up of this circuit, some design choices were made to improve simplicity of the circuit.

Removal of counter variable

Examples and recommendations featured two counter variables. Counter variables are used to keep track of repeated numbers in the sequence, for example, in the sequence 4005, the circuit would not know if 0 should go to 0 or 5 if there was no counter. Two counter variables are only needed if a digit repeats itself 3+ times (00 → 01 → 10). Since the sequence [redacted] only has the same digit appear at most twice, a flip flop could be removed from the set up.

Implementation of NOT gates

Different JK flip flops can be found in different labs. By added a NOT gate to K inputs, this allows for a more universal design because if inputs J/K were used instead of J/NOT K, these extra NOT gates could be removed, and the circuit would still perform as expected.

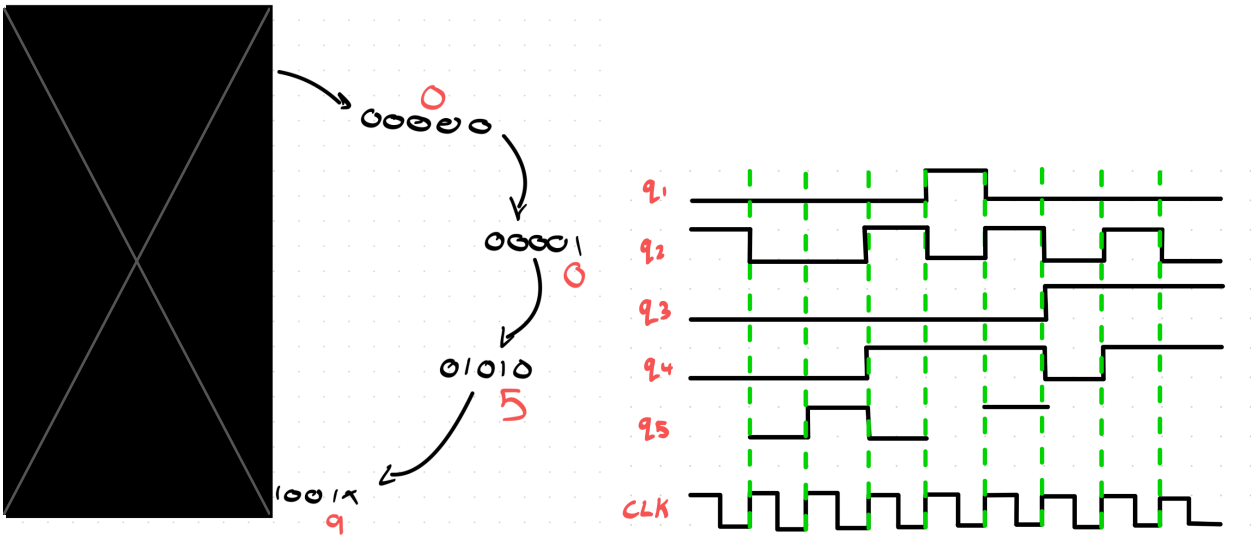
Rewiring K1 and K2 bits

At first glance, the simplest integration of K1 and K2 seem to be q1 and NOT q1, respectively. However, since neither has any bits which need to be 0 at any point, they both can be wired to HIGH (or LOW in the NOT K case).

Universal Set/Reset

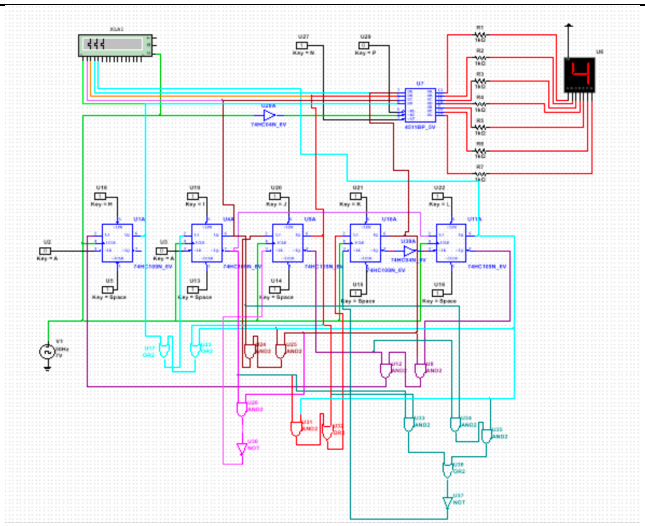
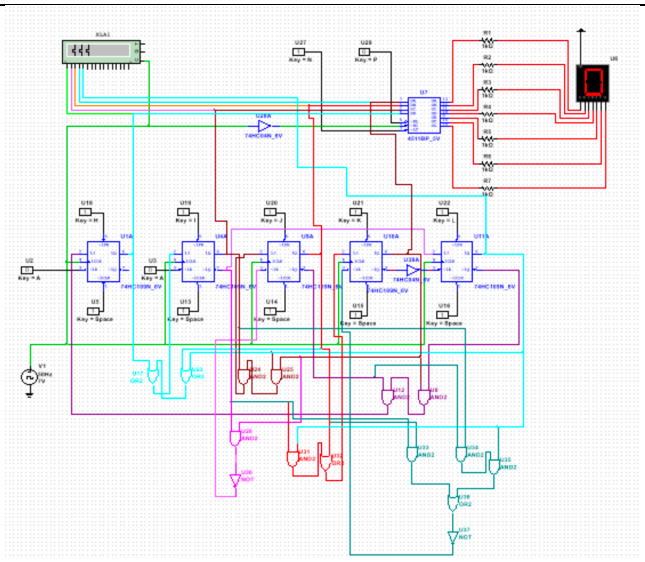
The state transition table only accounts for the digits; 0, 2, 3, 4, [redacted]. Additionally, the numbers other than 0 and 5 have an unknown counter state. This can lead to the circuit getting caught in different stable loops, giving the illusion of incorrect logic. By hooking up all the "clear" (or reset) inputs to a common point, a switch can be made and force the circuit into a 0 state. This is better than forcing the circuit into a state of 4 (the starting number) because it is not known whether counter should be 1 or 0. It is guaranteed to force the circuit to work as intended because both 0000(0) and 0000(1) have a place in the state transition table. *This is shown in Multisim, lack of switches made this impossible to implement in the physical apparatus.*

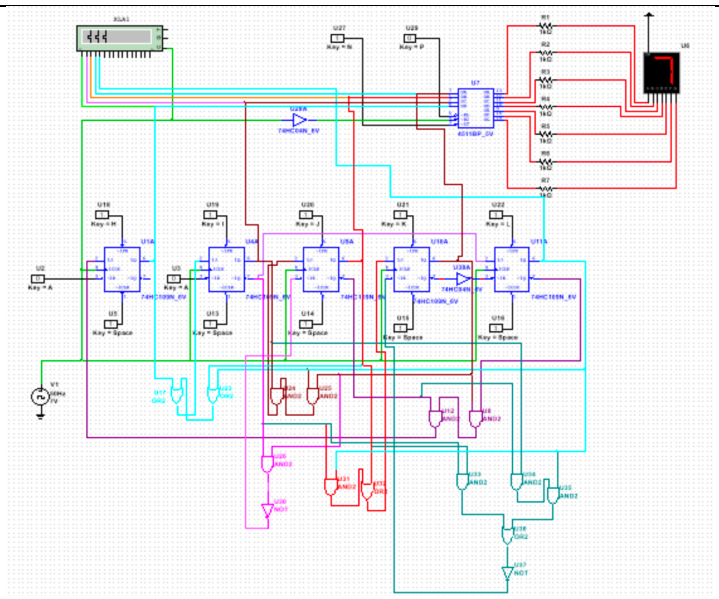
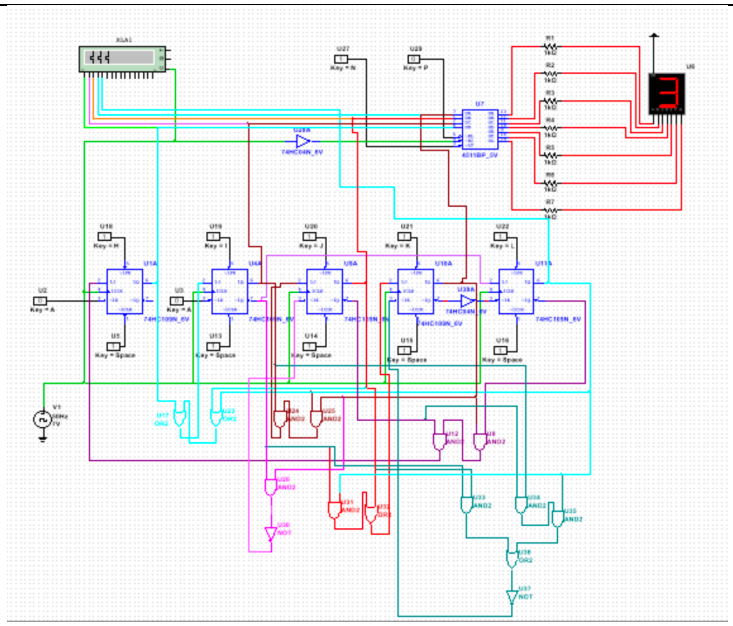
State Transition and Timing Diagrams

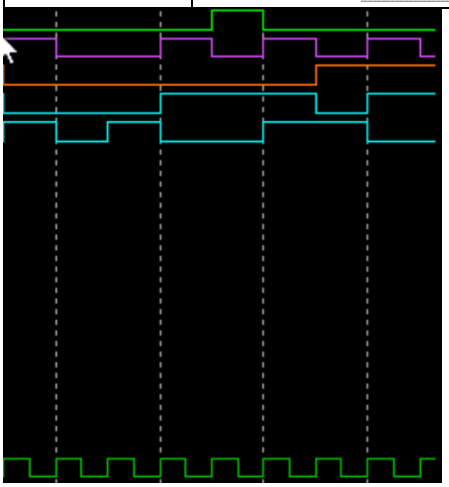


The state transition diagram shows the order of states which the circuit travels through as it completes one cycle. The timing diagram shows one cycle of the circuit starting at 4 and ending at 3. In both cases the exact state of q5 may be unknown, in the transition diagram it is marked as X and in the timing diagram it is left blank.

Multisim

Number	Multisim State
4	
0	

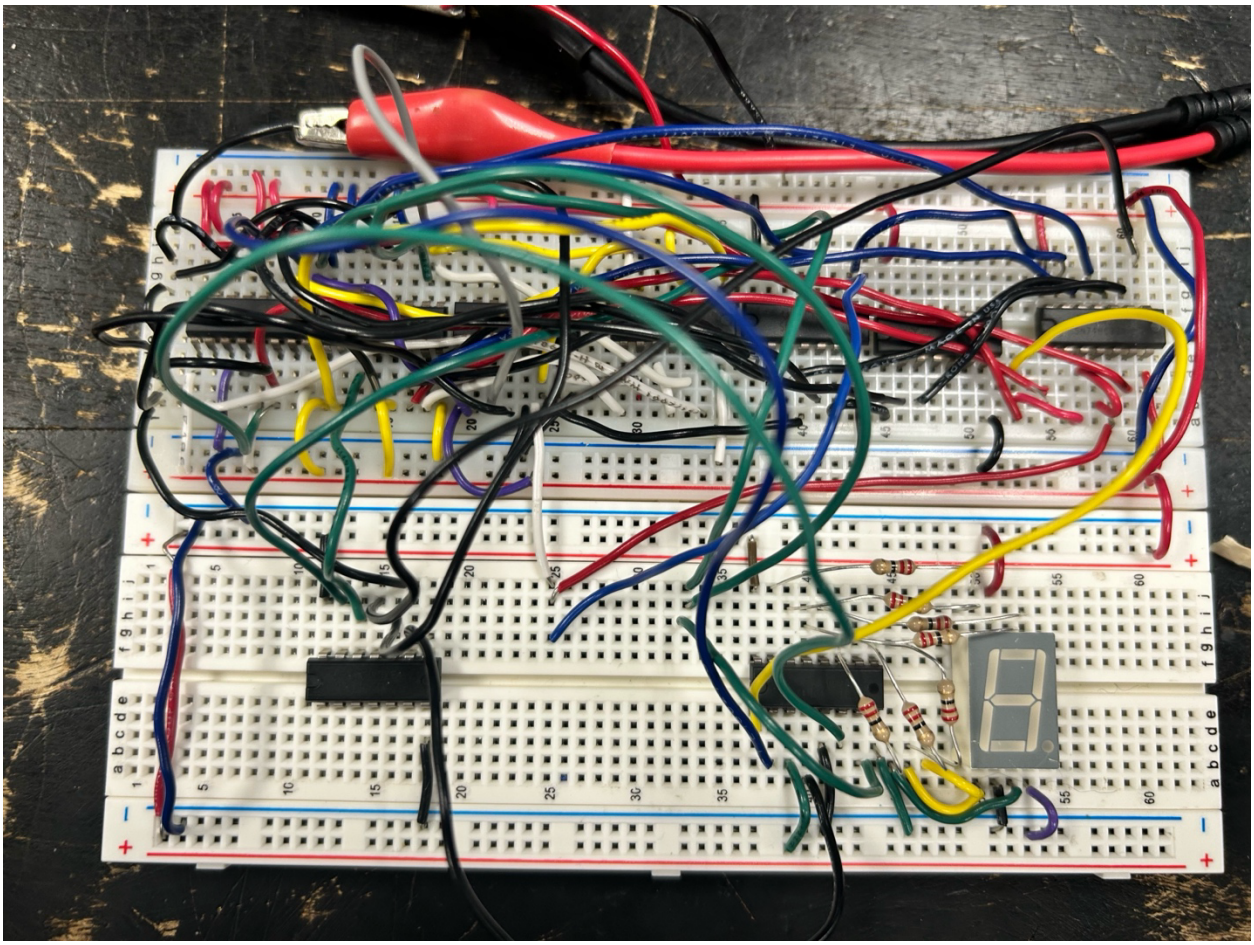
7	
3	



The timing diagram to the left shows the states of each bit from 4 → 3 and the clock at the bottom. In this case q5 also is defined for each tick, whereas when drawn by hand it was only defined where relevant.

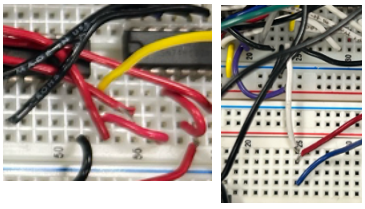
q1 is shown at the top, with q5 closer to the middle of the page and the clock at the bottom.

Physical Setup



Pictured above is the final circuit assembly. On the left side of the top breadboard, there are 3 74HC109 (JK flip flop) ICs, providing 6 flip flops, 5 of which were used. The 3 ICs to the right are 74HC08 (AND gates) and on the far right of the board is one 74HC32 (OR gate) IC. The left of the bottom of the board is a 74HC04 (NOT gate) IC and the IC in the bottom right is a 74HC4511E (binary decoder) IC which is connected to a LDS-C516R1 (7-segment) display.

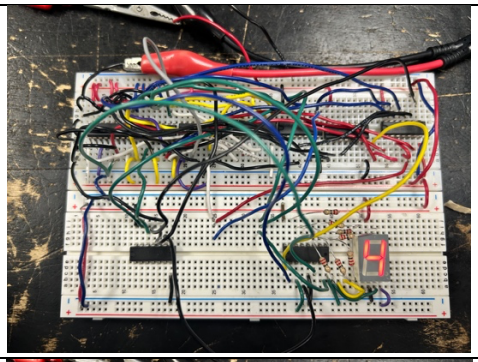
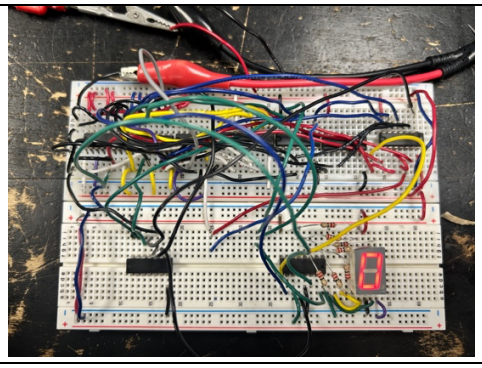
When connecting breadboards, it is important to carry ground and power inputs across power rails. Also, some outputs went to more locations that there are adjacent pins, in the event of this, an extra wire was used to act as an extension and provide room for more wires to easily attach.



To make the circuit as easy to dissect as possible, as many unique colors were used as possible. Due to a limited color range, some colors had to be reused. To counteract this, less complex gates were wired with repeat colors. The color which each gate is can be seen below.

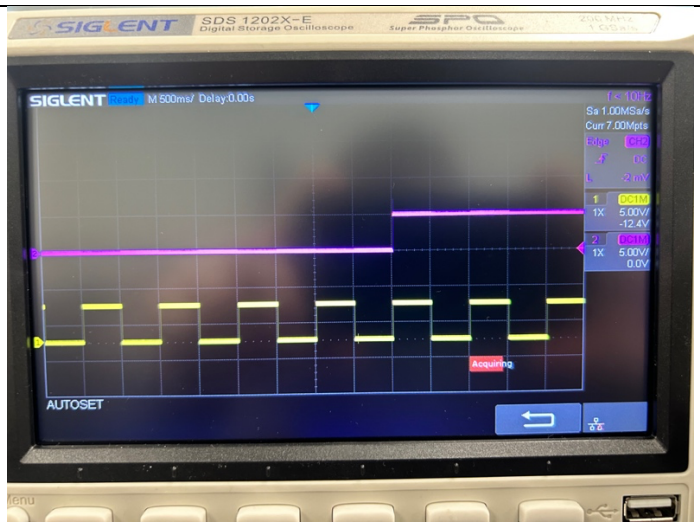
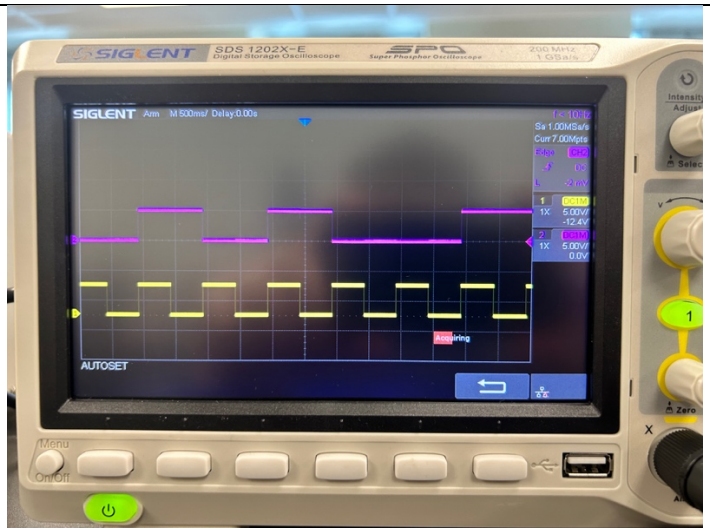
Breadboard Input	Wire Color Used
J1	White
!K1	Purple
J2	Red
!K2	Red
J3	Yellow
!K3	Green
J4	Blue
!K4	Black
J5	Purple
!K5	Grey
Clock	Black
VCC Input	Red
Ground Input	Black

Each number in the cycle can be seen below.

Number	7 Segment Output
4	
0	

The timing diagrams for Q1 → Q5 are pictured below, can be matched to the shown timing diagrams above. In the figure which features the hand drawn Q5 state, many clock cycles are labelled as "DON'T CARE", so they are left blank. Even though the philosophy remains true in the physical set up, each Q5 state will still have a high/low output for each number.

Output State	Timing Diagram
Q1	
Q2	

Q3	
Q4	
Q5	 *Note: Q5 starts on '3', the last digit of the sequence.*

Error Analysis

The process of creating the K-Maps was tedious rather than challenging, but focus was retained leading to no errors within the initial logic. When building the circuit on Multisim, the number of wires and their interactions with one another led to a couple errors in terms of wire mismanagement. This was resolved by color coding wires and carefully checking each J/K input and working backwards.

When building the circuit, error was avoided by testing each individual IC and cutting new wire. This meant that any issues that were to arise would be from wiring mistakes, as opposed to faulty components. Wires were also color coded, allowing for easier debugging. In the physical set up, there were issues at first, which stemmed from a single wire not being plugged in properly.

Conclusion

Starting a circuit from scratch was a unique experience and not without set back. Doing the necessary K-maping and finding the state transition table on excel was a large benefit as the work was much easier to follow and mitigated the chance for error.

Building the circuit in Multisim was difficult as the number of gates and wires led to multiple mistakes. Carefully double checking each wiring set up and Boolean expression helped resolve these issues. Additionally, checking the timing diagram can give insight into functionality of the circuit. For example, the wires connected to the decoder were originally attached backwards, making it seem like there were logic errors. Upon analysis of the timing diagram, it could be noted that the bits appeared to be switching in the correct order, and a simple wiring correction was needed.

Building the circuit on a breadboard was also a challenge as elements go from 2D to 3D, making troubleshooting difficult. It is important to test individual gates and wires to eliminate the idea that they might be causing the problem. Using shorter wires also helps keep the system neat and prevent unnecessary entanglement.